
UNIT 13 OBJECT-ORIENTED DATABASE

Structure

Page No.

- 13.0 Introduction
- 13.1 Objectives
- 13.2 Why Object-Oriented Database?
 - 13.2.1 Limitations of Relational Databases
 - 13.2.2 The Need for Object-Oriented Databases
- 13.3 Object-Relational Database Systems
 - 13.3.1 Complex Data Types
 - 13.3.2 Types and Inheritances in SQL
 - 13.3.3 Additional Data Types of OOP in SQL
 - 13.3.4 Object Identity and Reference Type Using SQL
- 13.4 Object-Oriented Database Systems
 - 13.4.1 Object Model
 - 13.4.2 Object Definition Language
 - 13.4.3 Object Query Language
- 13.5 OODBMS Vs Object-Relational Database
- 13.6 Summary
- 13.7 Solutions/Answers

13.0 INTRODUCTION

Object-oriented software development methodologies have become very popular in the development of software systems. Database applications are the backbone of most of these commercial business software developments. Therefore, it is but natural that object technologies also have their impact on database applications. Database models are being enhanced in computer systems for developing complex applications. For example, a true hierarchical data representation like a generalisation hierarchy scheme in a relational database would require a number of tables but could be a very natural representation for an object-oriented system. Thus, object-oriented technologies have found their way into database technologies. The present-day commercial RDBMS supports the features of object orientation.

This unit introduces various features of object-oriented databases. In this unit, we shall discuss the need for object-oriented databases, the complex types used in object-oriented databases, how these may be supported by inheritance, etc. In addition, we also define object definition language (ODL) and object manipulation language (OML). We shall introduce the object-oriented and object-relational databases.

13.1 OBJECTIVES

After going through this unit, you should be able to:

- define the need for object-oriented databases;
- explain the concepts of complex data types;
- use SQL to define object-oriented concepts;
- familiarise yourself with object definition and query languages, and
- define object-relational and object-oriented databases.

13.2 WHY OBJECT-ORIENTED DATABASE?

An object-oriented database is used for complex data types. Such database applications require complex interrelationships among object hierarchies to be represented in database systems. These interrelationships are difficult to implement in relational systems. Let us discuss the need for object-oriented systems in advanced applications in more detail. However, first, let us discuss the weakness of relational database systems.

13.2.1 Limitations of Relational Databases

Relational database technology was not able to handle complex application systems such as Computer-Aided Design (CAD), Computer Aided Manufacturing (CAM), Computer Integrated Manufacturing (CIM), Computer Aided Software Engineering (CASE) etc. The limitation for relational databases is that they have been designed to represent entities and relationships in the form of two-dimensional tables. Any complex interrelationship, like multi-valued attributes or composite attributes, may result in the decomposition of a table into several tables; similarly, complex interrelationships result in a number of tables being created. Thus, the main asset of relational databases, viz., its simplicity for such applications, is also one of its weaknesses in the case of complex applications.

The data domains in a relational system can be represented in relational databases as standard data types defined in the SQL. However, the relational model does not allow extending these data types or creating the user's own data types. Thus, limiting the types of data that may be represented using relational databases.

Another major weakness of the RDMS is that concepts like inheritance/hierarchy need to be represented with a series of tables with the required referential constraint. Thus, they are not very natural for objects requiring inheritance or hierarchy.

However, one must remember that relational databases have proved to be commercially successful for text-based applications and have lots of standard features, including security, reliability and easy access. Thus, even though they may not be a very natural choice for certain applications, yet their advantages are far too many. Thus, many commercial DBMS products are basically relational but also support object-oriented concepts.

13.2.2 The Need for Object-Oriented Databases

As discussed in the earlier section, relational database management systems have certain limitations. But how can we overcome such limitations? Let us discuss some of the basic issues with respect to object-oriented databases.

The objects may be complex, or they may consist of low-level objects (for example, a window object may consist of many simpler objects like a menu bar, scroll bar, etc.). However, to represent the data of these complex objects through relational database models you would require many tables – at least one each for each inherited class and a table for the base class. To ensure that these tables operate correctly, you would need to set up referential integrity constraints as well. On the other hand, object-oriented models would represent such a system very naturally through, an inheritance hierarchy. Thus, it is a very natural choice for such complex objects.

Consider a situation where you want to design a class (say an address class, which includes address lines, City, State and Pin code); the advantage of object-oriented

database management for such situations would be that they allow the representation of not only the structure but also the operation on newer user-defined database type such as finding the Address with similar Pin code. Thus, object-oriented database technologies are ideal for implementing such systems that support complex inherited objects, user defined data types (that define operations including the operations to support inheritance and polymorphism).

Another major reason for the need of object-oriented database system would be the seamless integration of this database technology with object-oriented applications. Software design is now mostly based on object-oriented technologies. Thus, object-oriented databases may provide a seamless interface for combining the two technologies.

Object-oriented databases are also required to manage complex, highly interrelated information. They provide solutions in the most natural and easy way that is closer to our understanding of the system. **Michael Brodie** related object-oriented systems to the human conceptualisation of a problem domain, which enhances communication among the system designers, domain experts, and the system end users.

The concept of an object-oriented database was introduced in the late 1970s, however, it became significant only in the early 1980s. The initial commercial product offerings appeared in the late 1980s. Some of the popular object-oriented database management systems were Objectivity/DB (developed by Objectivity, Inc.), VERSANT (developed by Versant Object Technology Corp.), Cache, ZODB, etc. An object-oriented database can be used in application areas such as e-commerce, engineering product data management, securities, medicine, etc.

Figure 1 traces the evolution of object-oriented databases. *Figure 2* highlights the strengths of object-oriented programming and relational database technologies. An object-oriented database system needs to capture the features from both these worlds. Some of the major concerns of object-oriented database technologies include access optimisation, integrity enforcement, archive, backup and recovery operations etc.

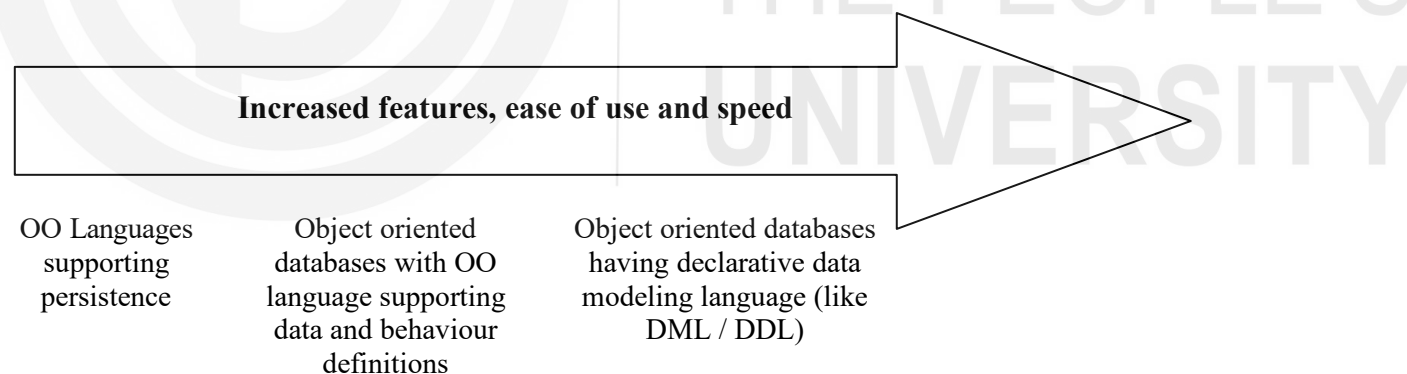


Figure 1: The evolution of object-oriented systems

Now, the question is, how does one implement an Object-oriented database system? As shown in *Figure 2* an object-oriented database system needs to include the features of object-oriented programming and relational database systems. Thus, the two most natural ways of implementing them will be either to extend the concept of object-oriented programming to include database features (OODBMS) or extend the relational database technology to include object-oriented features (Object Relational Database Systems). Let us discuss these two viz., the object relational and object-oriented database systems in more detail in the subsequent sections.

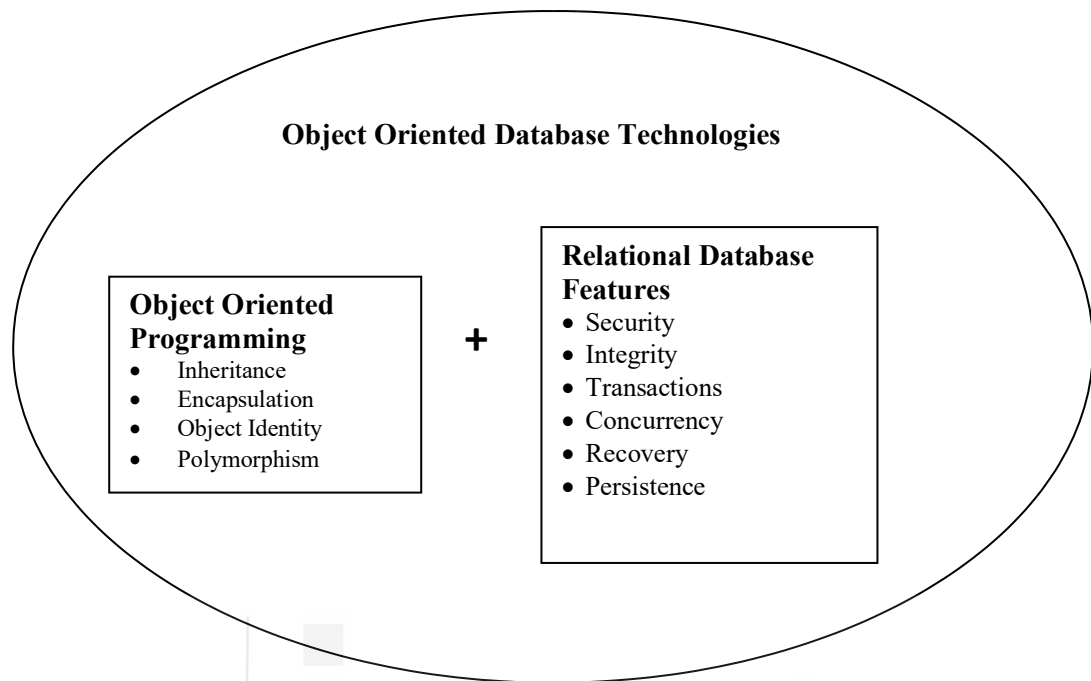


Figure 2: Makeup of an Object-Oriented Database System

13.3 OBJECT-RELATIONAL DATABASE SYSTEMS

Object-Relational Database Systems are the relational database systems that have been enhanced to include the features of object-oriented paradigm. This section provides details on how these newer features have been implemented in the SQL. Some of the basic object-oriented concepts that have been discussed in this section in the context of their inclusion into SQL standards include, the complex types, inheritance and object identity and reference types.

13.3.1 Complex Data Types

In the previous section, we have used the term complex data types without defining it. Let us explain this with the help of a simple example. Consider a composite attribute *Address*. The Address of a person in an RDBMS can be represented using the following:

House-no and apartment
Locality
City
State
Pin-code

When using RDBMS, such information either needs to be represented as separate attributes, as shown above, or just one string separated by comma or semicolon. The second approach is very inflexible, as it would require complex string related operations for extracting information. It also hides the details of an address; thus, it is not suitable.

If you represent the attributes of the *Address* as separate attributes, then the problem would be with respect to writing queries. For example, if you need to find the address of a person, you need to specify all the attributes that you have created for the Address

viz., House-no, Locality.... etc. The question is - Is there any better way of representing such information using a single field? If there is such a mode of representation, then that representation should permit distinguishing each element of the *Address*. The following may be one such possible solution:

```
CREATE TYPE Address AS
(
    Addressline1      Char(20)
    Addressline2      Char(20)
    City              Char(12)
    State             Char(15)
    Pincode           Char(6)
) ;
```

Thus, *Address* is now a new type that can be used while creating a database system schema as:

```
CREATE TABLE STUDENT
(
    name              Char(25)
    address           Address
    phone            Char(12)
    programme        Char(5)
    dob              Date
) ;
```

But what are the advantages of such definition?

Consider the following queries:

Find the name and address of the students enrolled in the PGDCA programme.

```
SELECT    name, address
FROM      student
WHERE     programme = 'PGDCA' ;
```

Please note that the attribute 'address', although composite, is used as a single attribute in the query. But can you also refer to individual components of this attribute?

Find the name and address of all the PGDCA students of Mumbai.

```
SELECT name, address
FROM    student
WHERE   programme = 'MCA' AND address.city='Mumbai' ;
```

Thus, such definitions allow you to handle a composite attribute as a single attribute with a user-defined type. You can also refer to any of the components of this attribute without any problems, so the data definition of the components of the composite attribute is still intact.

Complex data types also allow you to model a table with multi-valued attributes, which would require a new table in a relational database design. For example, a library database system would require representing the following information of a book.

- ISBN number
- Book title
- Authors
- Published by
- Subject areas of the book.

Clearly, in the table above, authors and subject areas are multi-valued attributes. You

need to design two relational database tables for these attributes – Author (ISBNnumber, author) and Area (ISBNnumber, subject area). (Please note that this design does not consider the author's position in the list of authors).

Although this database solves the immediate problem, yet it is a complex design. An object-oriented database system may solve this problem. This is explained in the next section.

13.3.2 Types and Inheritances in SQL

In the previous sub-section, we discussed the data type – *Address*. It is a good example of a structured type. In this section, let us give more examples for such types, using SQL. Consider the attribute:

- Name – that includes given name, middle name and surname.
- Address – that includes address details, city, state and pin code.

These types can be defined using SQL extensions, as given below:

```
CREATE TYPE Name AS (  
    Given_name Char (20),  
    Middle_name Char (15),  
    Sur_name Char (20)  
)  
FINAL
```

This type/class cannot be inherited further due to the keyword FINAL.

```
CREATE TYPE Address AS (  
    Add_det Char(20),  
    City Char(20),  
    State Char(20),  
    Pincode Char(6)  
)  
NOT FINAL
```

You can use this class to create inherited classes, like *Home_Address* and *Office_Address*, as this type/class is NOT FINAL.

The FINAL and NOT FINAL keywords have the same meaning as you learned in JAVA, i.e., a FINAL class cannot be inherited further.

These types can now be used to create a student class, which has data members and methods that work on objects of the student class, as follows:

```
CREATE TYPE Student AS (  
    name Name,  
    address Address,  
    dob Date  
)  
NOT FINAL  
METHOD ageinyears (givendate Date)  
    RETURN INTERVAL YEAR;
```

The method can be implemented separately using the following SQL Commands:

```

CREATE INSTANCE METHOD (givendate Date)
    RETURN INTERVAL YEAR
FOR Student
begin
    Return (givendate - self.dob);
end

```

This method computes the age on a given date. Please note that Date is a data type of SQL. FOR is the looping construct and will result in the execution of this method for every student object's instance.

The possibility of using constructors also exists, but a detailed discussion on that is beyond the scope of this unit.

Type Inheritance

In the present standard of SQL, you can define inheritance. Let us explain this with the help of an example.

Consider a type *University_person* defined as:

```

CREATE TYPE      University_person AS (
    name         Name,
    address      Address
)

```

Now, this type can be inherited by the *Staff* type or the *Student* type. For example, the *Student* type, if inherited from the class given above, would be:

```

CREATE TYPE      Student
UNDER            University_person (
    programme    Char(10),
    dob          Date
)

```

Similarly, you can create a sub-class for the *Staff* of the University as:

```

CREATE TYPE      Staff
UNDER            University_person (
    designation   Char(10),
    basic_salary  Number(7)
)

```

Notice that both the inherited types shown above inherit the *name* and *address* attributes from the type *University_person*. Methods can also be inherited in a similar way; however, they can be overridden if the need arises.

Table Inheritance

The concept of table inheritance has evolved to incorporate implementation of generalisation/ specialisation hierarchy of an E-R diagram. SQL allows inheritance of tables. Once a new type is declared, it could be used in the process of creation of new tables with the usage of keyword "OF". Let us explain this with the help of an example. Consider the classes *University_person*, *Staff* and *Student*, as we have defined in the previous sub-section. You can create the table for the type *University_person* as:

```

CREATE TABLE university_members OF University_person;

```

The table inheritance would allow us to create sub-tables for such tables as:

```
CREATE TABLE student_list OF Student
    UNDER university_members;
```

Similarly, you can create a table for the staff as:

```
CREATE TABLE staff OF Staff
    UNDER university_members;
```

Please note the following points for table inheritance:

- The type that is associated with the sub-table must be the sub-type of the type of the parent table. This is a major requirement for table inheritance.
- All the attributes of the parent table – (university_members in our case) should be present in the inherited tables.
- Also, the three tables may be handled separately. However, any record present in the inherited tables is also implicitly present in the base table. For example, any record inserted in the student_list table will be implicitly present in university_members tables.
- A query on the parent table (such as university_members) would find the records from the parent table and all the inherited tables (in our case all three tables). However, the attributes of the result table would be the same as the attributes of the parent table.
- You can restrict your query to only the parent table by using the keyword ONLY. For example,

```
SELECT name FROM university_members ONLY;
```

13.3.3 Additional Data Types of OOP in SQL

The object-oriented/relational database must support the data types that allow multi-valued attributes to be represented easily. Two such data types that exist in SQL are:

- Arrays – stores information in an order and
- Multisets – stores information in an unordered set.

Let us explain this with the help of an example of a book database as introduced in the section 13.3.1. A *Book* type can be represented using SQL as:

```
CREATE TYPE Book AS (
    ISBNNO Char (14),
    BOOK_TITLE Char (25),
    AUTHORS Char (25) ARRAY [5],
    PUBLISHED_BY Char (20),
    KEYWORDS Char (10) MULTiset
);
```

Please note, the use of the type ARRAY. Arrays not only allow authors to be represented but also allow the sequencing of the authors' names. Multiset allows a number of keywords without any ordering imposed on them.

But how can you enter data and query such data types? The following SQL commands would help in defining such a situation. But first, you need to create a table:

```
CREATE TABLE library OF Book;
INSERT INTO library VALUES ('008-124476-x', 'Database
Systems', ARRAY ['Silberschatz', 'Elmasri'],
'XYZ Publication', MULTiset [ 'Database',
```

```
'Relational', 'Object Oriented']) ;
```

The command above would insert information on a hypothetical book into the database. Let us now write a few queries on this database:

Find the list of books related to area Object Oriented:

```
SELECT ISBNNO, BOOK_TITLE
FROM library
WHERE 'Object Oriented' IN (UNNEST (KEYWORDS));
```

Find the first author of each book:

```
SELECT ISBNNO, BOOK_TITLE, AUTHORS[1]
FROM library ;
```

You can create many such queries. A detailed discussion on this can be found in the latest SQL standards and is beyond the scope of this unit.

13.3.4 Object Identity and Reference Type Using SQL

Till now, we have created the tables, but what about the situation when we have attributes that draw a reference to another attribute in the same table? This is a referential constraint. Thus, an object-relational database system should address the following two concerns:

- In relational databases, foreign keys are used to link the attributes in two different relations. Can such keys be used in an object-relational database?
- How will you identify the object that is being referenced?

The following example will address the concerns stated above.

Example: A library purchases books. Each book is given a unique book number called the catalogue number. The library maintains a procurement table, which can be created using the following SQL command:

```
CREATE TABLE procurement (
    CATALOGUE_NO CHAR (5),
    ISBNNO REF (Book) SCOPE (library)
) ;
```

The SQL statement given above would create a procurement table, which would assign two basic information to a newly purchased book – first, it will give the book a unique CATALOGUE_NO, and second, it will link this book to a specific record in the library table through an ISBNNO.

To insert a new book in this table, you must first create a Book object using the ISBNNO of this book. Assuming that such an object already exists, then you may use the following command to add a book to the procurement table:

```
INSERT INTO procurement
VALUES ('98765', NULL) ;
```

This command will add a new book to the procurement table having CATALOGUE_NO as '98765' and no reference to the ISBN number. The link to the ISBN number record will be created using the following SQL command:

```
UPDATE procurement
```

```

SET ISBNNO = (SELECT book_id
FROM library
WHERE ISBNNO = '83-7758-476-6')
WHERE CATALOGUE_NO = '98765';

```

Please note that in the query above, the sub-query generates the object identifier (book_id) for the ISBNNO of the book whose accession number is 98765. It then sets the reference for the desired record in the procurement.

This is a slightly complex procedure, and several other mechanisms exist to perform this operation. You can refer to them in the further readings.

Check Your Progress – 1

1) What is the need for object-oriented databases?

.....

.....

.....

2) How will you represent a complex data type?

.....

.....

.....

3) Represent an address using SQL that has a method for locating pin-code information.

.....

.....

.....

4) Create a table using the type created in question 3 above.

.....

.....

.....

5) How can you establish a relationship with multiple tables?

.....

.....

.....

13.4 OBJECT-ORIENTED DATABASE SYSTEMS

Object-oriented database systems are applying object-oriented concepts into database system models to create an object-oriented database model. This section describes the concepts of the object model, followed by a discussion on object definition and object manipulation languages that are derived from SQL.

13.4.1 Object Model

Object Data Management Group (ODMG) has designed the object model for the object-oriented database management system. The Object Definition Language (ODL) and Object Manipulation Language (OML) are based on this object model. Let us briefly define the concepts and terminology related to the object model.

Objects and Literal: These are the basic building elements of the object model. An object has the following four characteristics:

- A unique identifier
- A name
- A lifetime, defining whether it is persistent or not, and
- A structure that may be created using a type constructor. The structure in OODBMS can be classified as atomic or collection objects (like Set, List, Array, etc.).

A literal does not have an identifier but has a value that may be constant. The structure of a literal does not change. Literals can be atomic, such that they correspond to basic data types like int, short, long, float, etc. or structured literals (for example, current date, time, etc.) or collection literal defining values for some collection object.

Interface: Interfaces define the operations that can be inherited by a user-defined object. Interfaces are non-instantiable. All objects inherit basic operations (like copy object, delete object) from the interface of Objects. A collection object inherits operations – such as an operation to determine an empty collection – from the basic collection interface.

Atomic Objects: An atomic object is an object that is not of a collection type. They are user-defined objects that are specified using class keywords. The properties of an atomic object can be defined by its attributes and relationships. An example is the book object given in the next subsection. **Please note** here that a *class* is instantiable.

Inheritance: The interfaces specify the abstract operations that can be inherited by classes. This is called behavioural inheritance and is represented using the “:” symbol. Sub-classes can inherit the state and behaviour of super-class(s) using the keyword EXTENDS.

Extents: An extent of an object contains all the persistent objects of that class. A class having an extent can have a key.

In the following section, we shall discuss the use of the ODL and OML to implement object models.

13.4.2 Object Definition Language

Object Definition Language (ODL) is a standard language on the same lines as the DDL of SQL, that is used to represent the structure of an object-oriented database. It uses unique object identity (OID) for each object such as library item, student, account, fees, inventory etc. In this language, objects are treated as records. Any class in the design process has three properties that are attribute, relationship, and methods. A class in ODL is described using the following syntax:

```
class <name>
{
  <list of properties>
};
```

Here, *class* is a keyword, and the properties may be attributes, methods or relationships. The attributes defined in ODL specify the features of an object. It could be simple, enumerated, structured or complex type.

```

class Book
{
    attribute string ISBNNO;
    attribute string BOOKTITLE;
    attribute enum CATEGORY
        {text,reference,journal} BOOKTYPE;
    attribute struct AUTHORS
        {string fauthor, string sauthor,
         string tauthor} AUTHORLIST;
};

```

Please note that, in this case, we have defined authors as a structure and a new field on Book type as an enum.

These books need to be issued to the students. For that, you need to specify a relationship. The relationship defined in ODL specifies the method of connecting one object to another. You can specify the relationship by using the keyword “relationship”. For example, to connect a student object with a book object, you need to specify the relationship in the student class as:

```

relationship set <Book> receives

```

Here, for each object of the class student, there is a reference to the book object and the set of references is called *receives*.

But if we want to access the student based on the book, then the “inverse relationship” could be specified as

```

relationship set <Student> receivedby

```

You can specify the connection between the relationship *receives* and *receivedby* by using the keyword “inverse” in each declaration. If the relationship is in a different class, it is referred to by the relationship name followed by a scope resolution operator (::) and the name of the other relationship.

The relationship could be specified as:

```

class Book
{
    attribute string ISBNNO;
    attribute string BOOKTITLE;
    attribute integer PRICE;
    attribute string PUBLISHEDBY;
    attribute enum CATEGORY
        {text,reference} BOOKTYPE;
    attribute struct AUTHORS
        {string fauthor
         string sauthor
         string tauthor} AUTHORLIST;
    relationship set <Student> receivedby
        inverse Student::receives;
    relationship set <Supplier> suppliedby
        inverse Supplier::supplies;
};
class Student
{
    attribute string ENROLMENT_NO;

```

```

        attribute string NAME;
        attribute integer MARKS;
        attribute string COURSE;
        relationship set <Book> receives
            inverse Book::receivedby;
    };
class Supplier
{
    attribute string SUPPLIER_ID;
    attribute string SUPPLIER_NAME;
    attribute string SUPPLIER_ADDRESS;
    attribute string SUPPLIER_CITY;
    relationship set <Book> supplies
        inverse Book::suppliedby;
};

```

Methods could be specified with the classes along with input/output types. These declarations are called “signatures”. These method parameters could be *in*, *out* or *inout*. Here, the ‘*in*’ parameter is passed by value, whereas the ‘*out*’ or ‘*inout*’ parameters are passed by reference. Exceptions could also be associated with these methods.

```

class Student
{
    attribute string ENROLMENT_NO;
    attribute string NAME;
    attribute string st_address;
    relationship set <Book> receives
        inverse Book::receivedby;
    void findcity(in set<string>,out set<string>)
        raises (notfoundcity);
};

```

In the method findcity, the name of the city is passed with the objective to find the name of the students who belong to that specific city. In case blank is passed as a parameter for city name, then the exception notfoundcity is raised. The implementation of this method can be done separately.

The ODL could be atomic type or class names. The basic type uses many class constructors such as set, bag, list, array, dictionary and structure. We have shown the use of some in the example above. You can refer to the further readings for more detail on these.

Inheritance is implemented in ODL using subclasses with the keyword “extends”.

```

class Journal extends Book
{
    attribute string VOLUME;
    attribute string emailauthor1;
    attribute string emailauthor2;
};

```

Multiple inheritance is implemented by using extends separated by a colon (:). If there is a class Fee containing fee details, then multiple inheritance could be shown as:

```

class StudentFeeDetail extends Student:Fee
{
    void deposit(in set <float>, out set <float>)

```

```

        raises (refundToBeDone)
    };

```

Like the difference between relation schema and relation instance, ODL uses the class and its extent (set of existing objects). The objects are declared with the keyword “extent”.

```

class Student (extent firstStudent)
{
    attribute string ENROLMENT_NO;
    attribute string NAME;
    .....
};

```

It is not necessary in the case of ODL to define keys for a class. But if one or more attributes have to be declared as keys, then it may be done with the declaration of a key for a class with the keyword “key”.

```

class student (extent firstStudent key ENROLMENT_NO)
{
    attribute string ENROLMENT_NO;
    attribute string NAME;
    .....
};

```

Assuming that the ENROLMENT_NO and ACCESSION_NO form a key for the issue table, then:

```

class Issue (extent thisMonthIssue key
              (ENROLMENT_NO, ACCESSION_NO))
{
    attribute string ENROLMENT_NO;
    attribute string ACCESSION_NO;
    .....
};

```

The major considerations while converting ODL designs into relational designs are as follows:

- a) It is not essential to declare keys for a class in ODL, but in Relational design new attributes have to be created as a key.
- b) Attributes in ODL could be declared as non-atomic, whereas in Relational design they have to be converted into atomic attributes.
- c) Methods could be part of the design in ODL, but they cannot be directly converted into a relational schema, as they are not the property of a relational schema.
- d) Relationships are defined in inverse pairs for ODL, but in the case of relational design only one pair is defined.

For example, for the book class schema, the relation is:

```

Book (ISBNNO, TITLE, CATEGORY, fauthor, sauthor, tauthor)

```

ODL has been created with the features required to create an object-oriented database in OODBMS. You can refer to the further readings for more details on it.

13.4.3 Object Query Language

Object Query Language (OQL) is a standard query language that takes high-level declarative programming of SQL and object-oriented features of OOPs. Let us explain it with the help of examples.

Find the list of authors for the book titled “OODBMS”.

```
SELECT b.AUTHORS
FROM Book b
WHERE BOOK_TITLE="OODBMS" ;
```

Display the title of the book which has been issued to the student whose name is Anand.

```
SELECT BOOK_TITLE
FROM Book b, Student s
WHERE s.NAME ="Anand" ;
```

This query can also be written by using a relationship as:

```
SELECT BOOK_TITLE
FROM Book b
WHERE b.receivedby.NAME ="Anand" ;
```

In the previous case, the query creates a bag of strings, but when the keyword DISTINCT is used, the query returns a set.

```
SELECT DISTINCT BOOK_TITLE
FROM Book b
WHERE b.receivedby.NAME ="Anand" ;
```

When you add the ORDER BY clause to return a sorted list.

```
SELECT BOOK_TITLE
FROM Book b
WHERE b.receivedby.NAME ="Anand"
ORDER BY b.CATEGORY ;
```

Aggregate operators like SUM, AVG, COUNT, MAX, MIN could be used in OQL. If you want to compute the maximum marks obtained by any student, then the OQL command is

```
Max(SELECT s.MARKS FROM Student s);
```

Group By and Having clauses can also be used; however, you may refer to further readings for details on them.

Union, intersection and difference operators are applied to set or bag type with the keywords UNION, INTERSECT and EXCEPT. If you want to display the details of suppliers from PATNA and SURAT, then the OQL command is:

```
(SELECT DISTINCT su
FROM Supplier su
WHERE su.SUPPLIER_CITY="PATNA")
UNION
(SELECT DISTINCT su
FROM Supplier su
WHERE su.SUPPLIER_CITY="SURAT");
```

The result of the OQL expression could be assigned to host language variables. If costlyBooks is a set <Book> variable to store the list of books whose price is more than Rs. 500, then:

```
costlyBooks = SELECT DISTINCT b
              FROM Book b
              WHERE b.PRICE > 500
```

In this section, you have been introduced to OQL. You can refer to further readings for more details on OQL.

Check Your Progress – 2

- 1) Create a class staff using ODL that also references the Book class given in section 13.4.

.....

.....

- 2) What modifications would be needed in the Book class because of the table created by the above query?

.....

.....

- 3) Find the list of books that have been issued to “Shashi”.

.....

.....

.....

13.5 OODBMS VERSUS OBJECT RELATIONAL DATABASE

An object-oriented database management system is created on the basis of persistent programming paradigm, whereas an object-relational is built by creating object-oriented extensions of a relational system. In fact, both the products have clearly defined objectives. The following table shows the difference between them:

Object-Relational DBMS	Object-Oriented DBMS
The features of these DBMS include: • Support for complex data types • Powerful query languages support through SQL • Good protection of data against programming errors	The features of these DBMS include: • Support complex data types, • Very high integration of database with the programming language, • Very good performance • But not as powerful at querying as Relational.
One of the major assets here is SQL. Although, SQL is not as powerful as a Programming Language, but it is none-the-less essentially a fourth-generation language, thus, it provides excellent protection of data from the Programming errors.	It is based on object-oriented programming languages, thus, are very strong in programming, however, any error of a data type made by a programmer may affect many users.
The relational model has a very rich foundation for query optimisation, which helps in reducing the time taken to execute a query.	These databases are still evolving in this direction. They have reasonable systems in place.

These databases make the querying as simple as in relational even, for complex data types and multimedia data.	The querying is possible but somewhat difficult to get.
Although the strength of these DBMS is SQL, it is also one of the major weaknesses from the performance point of view of in memory applications.	Some applications that are primarily run in the RAM and require a large number of database accesses with high performance may find such DBMS more suitable. This is because of rich programming interface provided by such DBMS. However, such applications may not support very strong query capabilities. A typical example of one such application is databases required for CAD.

☛ Check Your Progress – 3

State True or False.

- 1) Object-relational databases cannot represent inheritance but can represent complex data types. T ☐ F ☐
- 2) The class extent defines the limit of a class. T ☐ F ☐
- 3) The query language of object-oriented DBMS is stronger than object-relational databases. T ☐ F ☐
- 4) SQL commands cannot be optimised. T ☐ F ☐
- 5) Object-oriented DBMS supports a very high level of integration of databases with OOP. T ☐ F ☐

13.6 SUMMARY

Object-oriented technologies are one of the most popular technologies in the present era. Object orientation has also found its way into database technologies. The object-oriented database systems allow the representation of user-defined types, including operation on these types. They also allow the representation of inheritance using the type and table inheritance. The idea here is to represent the whole range of newer types if needed. Such features help in enhancing the performance of a database application that would otherwise have many tables. SQL support these features for object-relational database systems.

The object definition languages and object query languages have been designed for the object-oriented DBMS on the same lines as that of SQL. These languages try to simplify various object-related representations using OODBMS.

The object-relational and object-oriented databases do not compete with each other but have different kinds of application areas. For example, relational and object-relational DBMS are most suited for simple transaction management systems, while OODBMS may find applications with e-commerce, CAD and other similar complex applications.

13.7 SOLUTIONS/ANSWERS

Check Your Progress 1

- 1) The object-oriented databases are need for:
 - Representing complex types.
 - Representing inheritance, polymorphism
 - Representing highly interrelated information
 - Providing object-oriented solution to databases bringing them closer to OOP.
- 2) Primarily by representing it as a single attribute. All its components should also be referenced separately.

3)

```
CREATE TYPE Addrtype AS
(
    houseNo    CHAR(8),
    street     CHAR(10),
    colony     CHAR(10),
    city       CHAR(8),
    state      CHAR(8),
    pincode    CHAR(6),
);

METHOD pin() RETURNS CHAR(6);
CREATE METHOD pin() RETURNS CHAR(6);
FOR Addrtype
BEGIN
    Return pincode;
END
```

- 4)
- ```
CREATE TABLE addresswithpin OF Addrtype
(
 REF IS addressid,
 PRIMARY KEY
 (street,colony,city,pincode)
);
```

- 5) The relationship can be established with multiple tables by specifying the keyword “SCOPE”. For example:

```
CREATE TABLE mylibrary
(
 mybook REF(Book) SCOPE library;
 myStudent REF(Student) SCOPE student;
 mySupplier REF(Supplier) SCOPE supplier;
);
```

### Check Your Progress 2

1)

```
class Staff
{
 attribute string STAFF_ID;
```

```
attribute string STAFF_NAME;
attribute string DESIGNATION;
relationship set <Book> issues
 inverse Book::issuedto;
};
```

- 2) The Book class needs to represent the relationship that is with the Staff class.

This would be added to it by using the following commands:

```
RELATIONSHIP SET < Staff > issuedto
 INVERSE :: issues Staff
```

- 3) 

```
SELECT DISTINCT b.TITLE
FROM BOOK b
WHERE b.issuedto.NAME = "Shashi"
```

### Check Your Progress 3

- 1) False 2) False 3) False 4) False 5) True



ignou  
THE PEOPLE'S  
UNIVERSITY